

Executive Summary

Long-horizon reinforcement tasks (e.g. multi-step QA or planning) with only sparse final rewards pose a severe credit-assignment challenge. Standard RL methods like Monte Carlo return or TD learning either propagate the delayed reward equally to all steps (high variance) or incrementally (bias), giving little signal to intermediate actions ¹. Recent work proposes diverse solutions: **reward redistribution** (RUDDER), **intrinsic per-step signals** (IGPO, SPRO/SPARK, PRInTS), **game-theoretic/token-level credit** (SCAR), and **process models or interventions** that break down trajectories (InT, MGR). These methods either learn or compute dense “intrinsic” rewards for steps so that the sum equals the final outcome, or reorganize trajectories (hindsight methods) to retroactively assign credit. In summary, we found multiple complementary approaches: RUDDER learns an LSTM return-decomposer to reassign delayed reward to contributory steps ²; IGPO defines per-turn rewards as the model’s *information gain* toward the true answer ³; SCAR uses Shapley-value decomposition to allocate final sequence reward to individual tokens ⁴; MGR gates trajectory reward by per-step validity to avoid rewarding invalid actions ⁵; GRPO- λ extends value-based traces to token sequences with λ -returns ⁶; InT identifies and corrects the first reasoning error and fine-tunes on that partial trajectory, localizing credit ⁷; SPARK trains a step-level reward model using synthetic verification (self-consistency) labels ⁸; and PRInTS scores each reasoning step by Monte Carlo *information gain* on the final answer ⁹. We compare these methods below and note their assumptions (e.g. reliance on reference answers, model type) and how they scale to very long LLM-generated trajectories.

Methods for Long-Horizon Credit Assignment

- **RUDDER: Return Decomposition for Delayed Rewards** (Arjona-Medina *et al.*, NeurIPS 2019). RUDDER trains an LSTM to predict the total return of a trajectory and then *decomposes* the return into contributions at each time step (via difference in successive LSTM predictions). This “return decomposition” redistributes the sparse final reward backward onto the most credit-worthy actions, effectively making expected future reward zero ². In practice RUDDER assigns immediate rewards to states/actions that maximally contributed to the final outcome, converting delayed credit into a supervised regression task ².
- **IGPO: Information Gain-based Policy Optimization** (Wang *et al.*, ICLR 2026). IGPO generates *dense intrinsic rewards* at each turn of a multi-step interaction by measuring the marginal increase in the model’s belief about the correct answer. Concretely, it treats each step as an information-acquisition process and sets the turn-level reward equal to the change in log-probability of the true answer under the current policy. These intrinsic “information-gain” rewards telescope over the interaction and, when summed, equal the overall increase in confidence in the answer ³. This creates fine-grained credit signals without any external supervision.
- **SCAR (Shapley Credit Assignment Rewards)** (Cao *et al.*, ICLR 2026). SCAR applies cooperative game-theoretic Shapley values to sequence-level rewards in RLHF. Given a sparse final reward for an entire generated sequence, SCAR computes each token’s Shapley value — its marginal contribution to the sequence reward — and uses that as a dense per-token reward. This principled decomposition

ensures “fair” credit to each token or span without needing an auxiliary critic or annotated step labels ⁴ .

- **MGR (Multiplicative Gated Rewards)** (Lu *et al.*, ICLR 2026). MGR decouples **local validity** from **global success** in LLM agent trajectories. It assumes each action has a simple binary validity signal (e.g. API call success). MGR computes each action’s reward as $Validity(t) \times |TotalReward|$, i.e. multiply the signed trajectory-level reward by +1 or -1 depending on local validity ⁵ . This way, an action only receives the final reward if it was locally valid, preventing an invalid step from undeserved positive credit. It still uses the sparse outcome reward, but gates it per step via a learned or factual validity flag.
- **GRPO- λ (λ -Return Extension for LLMs)** (Parthasarathi *et al.*, 2025, arXiv). This method augments Group-Relative Policy Optimization (GRPO) by incorporating eligibility-trace-style credit assignment. After a full trajectory is generated, GRPO- λ computes an approximate λ -return for each token step using the token-level log-probabilities and a critic-free TD-error. Effectively it distributes the final return backwards along the token sequence with exponentially decaying weights (like TD(λ)), providing intermediate reward estimates per token. Empirically this yields 30–40% faster RL training on math benchmarks than GRPO alone ⁶ .
- **InT (Self-Proposed Intervention Training)** (Yang *et al.*, 2026, arXiv). InT is a two-stage method using reference solutions to induce credit assignment. The model generates a full reasoning chain; if the final answer is wrong, it locates the first incorrect step and generates a one-step “intervention” (a correction). The training then fine-tunes the model on the partial trajectory up to that point plus the intervention, treating the correction as supervised signal. This localizes the blame to the exact error step. By continually correcting the first mistake in its chain of thought, the model learns fine-grained credit for which reasoning step needs fixing ⁷ .
- **SPARK (Stepwise Process-Aware Rewards)** (Rahman *et al.*, ICLR 2026). SPARK constructs a *process reward model* without ground-truth step labels. It first generates many candidate reasoning trajectories and uses an LLM-based verifier (via self-consistency and meta-critique) to judge correctness of each step. Those synthetic verifications are used to fine-tune a generative model that scores each intermediate step (a “process reward model”). At RL time, SPARK’s learned model provides dense step-level rewards. In effect, SPARK sidesteps the need for manual intermediate labels by using ensemble verification to distill a stepwise reward function ⁸ .
- **PRInTS (Reward Modeling for Information Seeking)** (Lee *et al.*, 2025, arXiv). PRInTS assigns credit by directly measuring each step’s “information gain.” For each reasoning step and tool action, it runs Monte Carlo rollouts from that step to estimate how much closer the agent gets to the correct answer (increase in success probability). A step is given positive reward if it materially boosts the chance of eventual success and negative reward if it misleads. Formally, PRInTS samples rollouts from each intermediate state and computes a gain score for that step; the sum of these gains telescopes to the final reward. Thus each step’s credit is the estimated long-run impact on the final outcome ⁹ .
- **Hindsight Credit Assignment (HCA)** (Harutyunyan *et al.*, NeurIPS 2019). HCA assigns credit by learning a *hindsight distribution* over actions. After observing a trajectory with total reward, HCA re-weights each taken action by how that action would have affected the probability of achieving the

observed return. Essentially it asks, “given this future outcome, which past actions were likely responsible?” and uses that distribution to update credit. In this way, credit is reassigned retrospectively based on each action’s estimated contribution to the return ¹⁰ .

- **Self-Critical Sequence Training (SCST)** (Rennie *et al.*, CVPR 2017). Although not new to RLHF, SCST is often used to train sequence models via policy gradients. SCST uses REINFORCE with a learned baseline: it samples a sequence, computes its reward (e.g. success or metric), and subtracts the reward of the model’s own greedy output. The gradient thus assigns credit only to the advantage of each sampled sequence over the baseline. This normalizes sequence-level returns but still treats the entire sequence monolithically—no fine-grained step signals are learned ¹¹ .

Method Comparison

Method	Intermediate supervision needed?	Model assumptions	Long-sequence scaling	Applicability to LLM trajectories
RUDDER (2019)	No (uses only final reward)	Any sequential model (e.g. RNN/ LSTM)	Good – explicitly targets long delays via LSTM decomposition	High (applies to any seq. task)
IGPO (2026)	No (intrinsic belief updates)	LLM-based agent with probabilistic outputs	Moderate – uses $O(T)$ rollouts per turn but linear in length	High (designed for LLM agents)
SCAR (2026)	No (uses final reward and token context)	Any generative sequence model	Moderate – requires computing Shapley values ($O(n^2)$ per token window, approximate)	High (token-level, for RLHF)
MGR (2026)	Yes: needs local validity signal	LLM agent with external tools (API outputs)	Good – simple $O(1)$ operation per step	Yes (if validity signal defined)
GRPO-λ (2025)	No (critic-free λ -return)	LLM for reasoning (trained with PPO)	Moderate – eligibility traces add overhead $O(T)$	High (works on math LLM tasks)
InT (2026)	Yes: ground-truth corrections	LLM tasks with available references	Poor without references (requires them)	Specific (math problems with answer)
SPARK (2026)	No (uses LLM verifier as pseudo-labels)	LLM generator + verifier model	Moderate – heavy pretraining but RL overhead similar to standard	High (LLM reasoning tasks)

Method	Intermediate supervision needed?	Model assumptions	Long-sequence scaling	Applicability to LLM trajectories
PRInTS (2025)	No (uses final answer only)	LLM agent + Monte Carlo simulator	Limited by MC rollouts per step ($O(n \cdot M)$)	High (LLM with tools for info seeking)
HCA (2019)	No (off-policy credit reweighting)	Markov decision processes	Good – uses importance sampling (DICE) for stability	Potentially (if formulated as MDP)
SCST (2017)	No (uses own baseline)	Any sequence model	Good – simple REINFORCE with baseline	Standard seq training (including LLM output)

Each method trades off different assumptions. RUDDER and HCA redistribute the final reward through learned or calculated hindsight distributions without extra labels. Methods like IGPO, SPARK, and PRInTS derive *intrinsic* step rewards from the model’s own beliefs or auxiliary computations. Shapley (SCAR) and intervention-based (InT) approaches explicitly apportion credit by game theory or by supervised corrections. Reward shaping methods (not tabulated) – e.g. potential-based shaping ² – could also inject intermediate reward but require hand-crafted potentials.

In practice, LM-specific methods (IGPO, SCAR, GRPO- λ , SPARK, PRInTS) are directly tailored to multi-turn generation or reasoning tasks, yielding dense per-step signals without human annotation. Others (RUDDER, HCA) come from general RL and demonstrate how any delayed reward can be retroactively spread over key actions. The table above summarizes key attributes: notably, most approaches avoid needing explicit intermediate labels (except InT or custom shaping), many assume a probabilistic LLM or MDP, and all aim to scale better than naive REINFORCE on very long trajectories.

¹ ² RUDDER: Return Decomposition for Delayed Rewards

<http://papers.neurips.cc/paper/9509-rudder-return-decomposition-for-delayed-rewards.pdf>

³ [2510.14967] Information Gain-based Policy Optimization: A Simple and Effective Approach for Multi-Turn Search Agents

<https://arxiv.org/abs/2510.14967>

⁴ SCAR: Shapley Credit Assignment for More Efficient RLHF | OpenReview

<https://openreview.net/forum?id=6OxvdqP6RH>

⁵ Both Local Validity and Global Effectiveness Matter: Decoupled Credit Assignment for Long-Horizon Agentic Learning | OpenReview

<https://openreview.net/forum?id=Suja4AQ9Fo>

⁶ [2510.00194] GRPO- λ : Credit Assignment improves LLM Reasoning

<https://arxiv.org/abs/2510.00194>

⁷ Paper page - InT: Self-Proposed Interventions Enable Credit Assignment in LLM Reasoning

<https://huggingface.co/papers/2601.14209>

8 SPARK: Stepwise Process-Aware Rewards for Reference-Free Reinforcement Learning | OpenReview
<https://openreview.net/forum?id=7y9IKjl8dt>

9 PRInTS: Reward Modeling for Long-Horizon Information Seeking
<https://arxiv.org/pdf/2511.19314>

10 openreview.net
<https://openreview.net/pdf/2efe3e8b51768f8e9e0cc2aad7eb4c52718e4812.pdf>

11 Self-Critical Sequence Training for Image Captioning
https://openaccess.thecvf.com/content_cvpr_2017/papers/Rennie_Self-Critical_Sequence_Training_CVPR_2017_paper.pdf